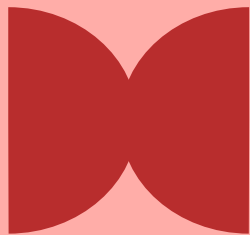


CSCI/MATH
608:
Data
Science for
Graduate
Students



Dr. Kathleen Isenegger
(she/her)

Week 5–Wednesday

Visualizing Data

- ❑ Choosing your visualization
- ❑ Conveying your message clearly
- ❑ Line plots
- ❑ Scatter plots
- ❑ Bar plots
- ❑ Histograms
- ❑ Chart layers
- ❑ Troubleshooting Categories
- ❑ Stacked Bars
- ❑ Facets
- ❑ Graphs and Trees
- ❑ Explaining visualizations
- ❑ Saving visualizations

Learning Goals

- ❑ Describe when to use the following kinds of visualizations to answer specific questions using a data set:
 - ❑ scatter plots
 - ❑ line plots
 - ❑ bar plots
 - ❑ histogram plots
- ❑ Given a data set and a question, select from the above plot types and use Python to create a visualization that best answers the question.
- ❑ Evaluate the effectiveness of a visualization and suggest improvements to better answer a given question.

Learning Goals

- ❑ Referring to the visualization, communicate the conclusions in non-technical terms.
- ❑ Identify rules of thumb for creating effective visualizations.
- ❑ Use the altair library in Python to create and refine the above visualizations using:
 - ❑ graphical marks: `mark_point`, `mark_line`, `mark_circle`, `mark_bar`, `mark_rule`
 - ❑ encoding channels: `x`, `y`, `color`, `shape`
 - ❑ labeling: `title`
 - ❑ transformations: `scale`
 - ❑ subplots: `facet`

Learning Goals

- ❑ Define the two key aspects of altair charts:
 - ❑ graphical marks
 - ❑ encoding channels
- ❑ Describe the difference in raster and vector output formats.
- ❑ Use `chart.save()` to save visualizations in `.png` and `.svg` format.

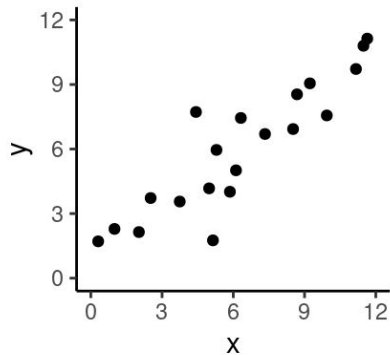
Choosing your Visualization

- ❑ The purpose of a visualization is to answer a question about data
- ❑ They can answer **descriptive** and **exploratory** questions

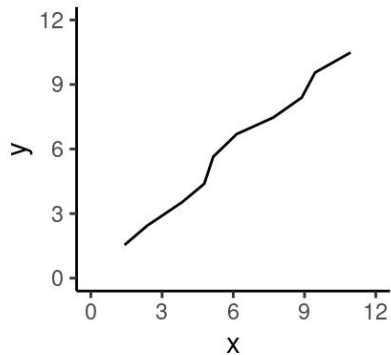
scatter plots	visualize the relationship between two quantitative variables
line plots	visualize trends with respect to an independent, ordered quantity (e.g., time)
bar plots	visualize comparisons of amounts
histograms	visualize the distribution of one quantitative variable (i.e., all its possible values and how often they occur)

Choosing your Visualization

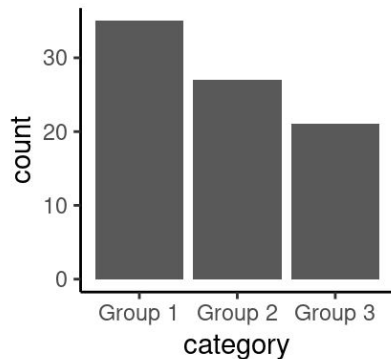
Scatter plot



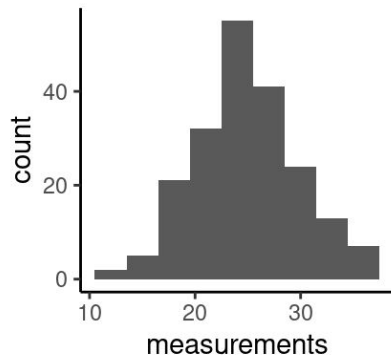
Line plot



Bar plot



Histogram



Choosing your Visualization

- ❑ Bar plots are better than pie charts (we can compare bar sizes better than slices)
- ❑ 3-D Visualization are hard to understand when converted to a 2-D format
- ❑ Bar plots are also better than tables (we can compare bar sizes better than number amounts)

Conveying your message clearly

- ❑ Reduce the mental load on your audience when viewing your visualization, making it easier for them to understand and remember your message quickly
- ❑ Answer the question simply and plainly
- ❑ Use legends and labels
- ❑ Ensure text, symbols, lines, etc. are big enough to be seen easily
- ❑ Ensure data is clearly visible and not hidden behind other objects (be wary of overplotting)

Conveying your message clearly

- ❑ Make sure to use color schemes that are understandable by those with colorblindness
- ❑ Redundancy can be helpful; sometimes conveying the same message in multiple ways reinforces it
- ❑ Use colors sparingly. Too many different colors can be distracting and create false patterns
- ❑ Only make the plot area as large as needed
- ❑ Don't adjust the axes to zoom in on small differences. If the difference is small, show that!

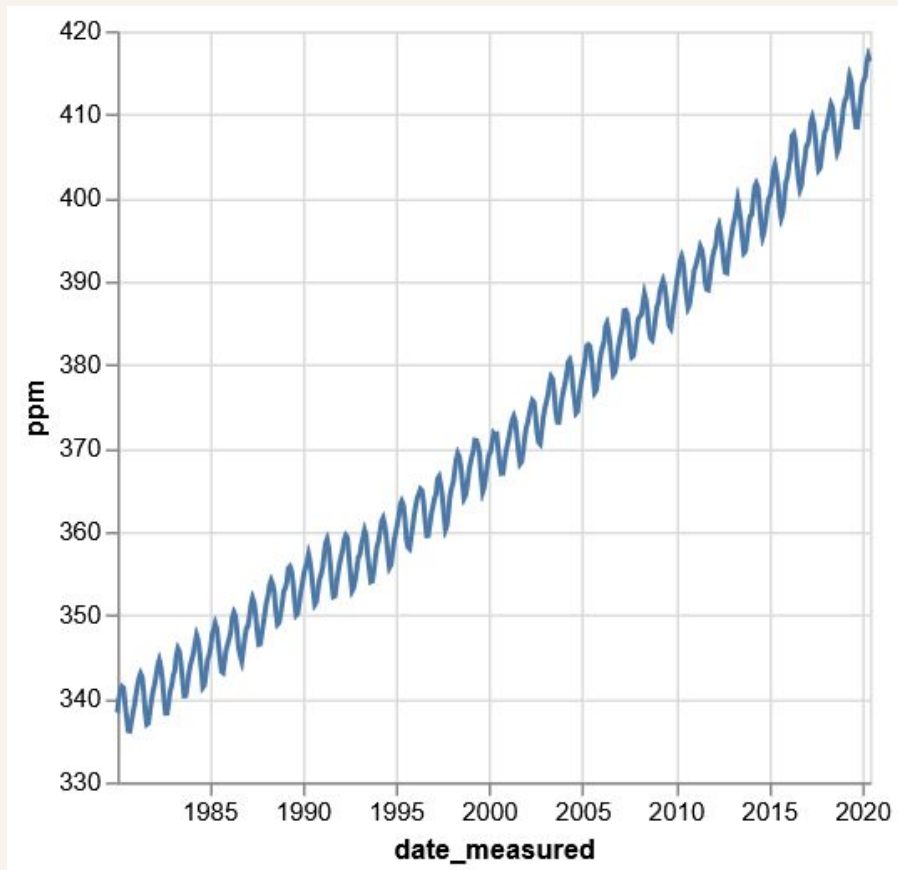
Line Plots

- ❑ To create a scatter plot with altair, we specify:
 - ❑ Dataframe to visualize
 - ❑ Graphical mark
 - ❑ Encoding channels, to map columns in the dataframe to parts of the visualization

```
co2_line = alt.Chart(co2_df).mark_line().encode(  
    x=alt.X("date_measured"),  
    y=alt.Y("ppm").scale(zero=False)  
)
```

- ❑ We specify that the y-axis scale doesn't have to start at zero

Line Plots



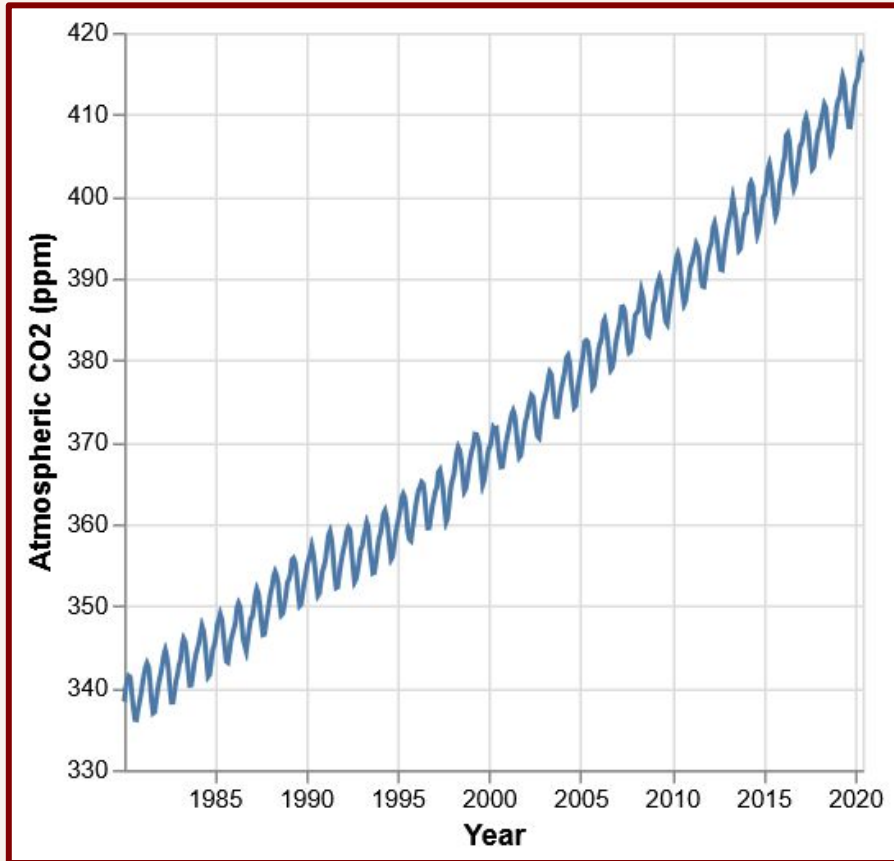
Line Plots

- ❑ We can improve the clarity
 - ❑ Add axis titles
 - ❑ Configure title font size

```
co2_line_labels = alt.Chart(co2_df).mark_line().encode(  
    x=alt.X("date_measured").title("Year"),  
    y=alt.Y("ppm").scale(zero=False).title("Atmospheric CO2 (ppm)")  
) .configure_axis(titleFontSize=12)
```

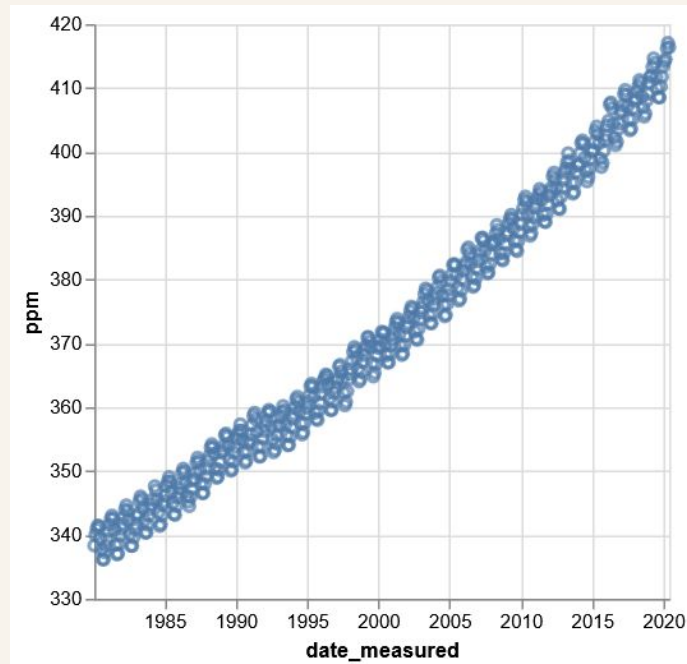
- ❑ You can pass a list of strings as the title to create a multi-line title

Line Plots



Scatter Plots

- ❑ To turn a line plot into a scatter plot, just replace **mark_point** with **mark_line**



```
co2_line = alt.Chart(co2_df).mark_point().encode(  
    x=alt.X("date_measured"),  
    y=alt.Y("ppm").scale(zero=False)  
)
```

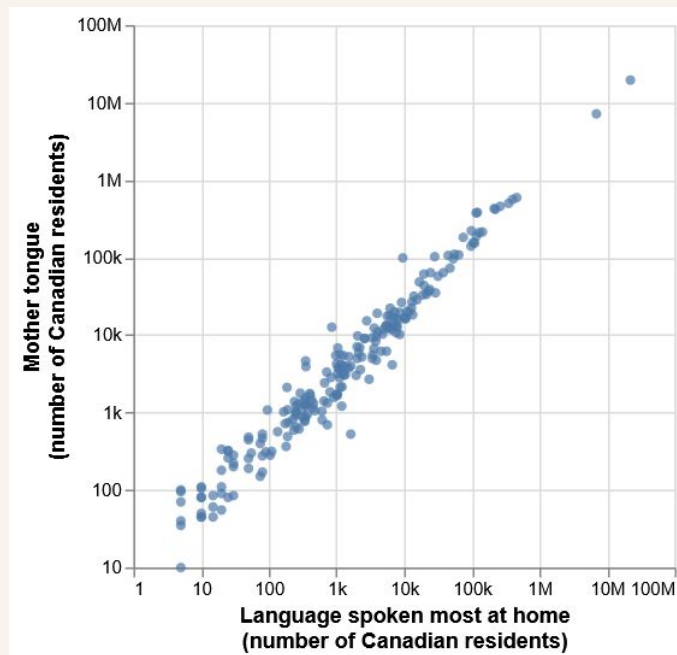
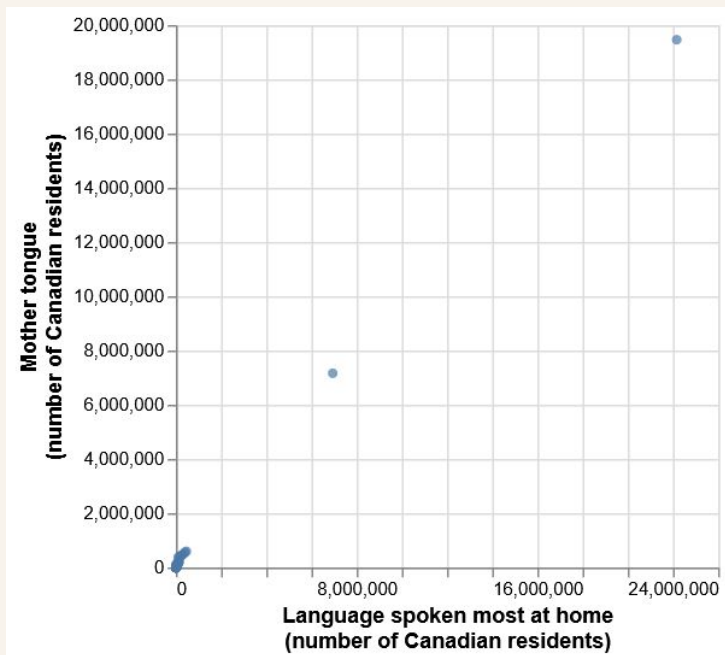
Scatter Plots

- ❏ We can also change the size and color of the data markers

```
co2_line = alt.Chart(co2_df).mark_point(size=10, color="black").encode(  
    x=alt.X("date_measured"),  
    y=alt.Y("ppm").scale(zero=False)  
)
```


Logarithmic Scaling

- ❑ When you have both really small and really large values, a logarithmic scale makes it clearer to see

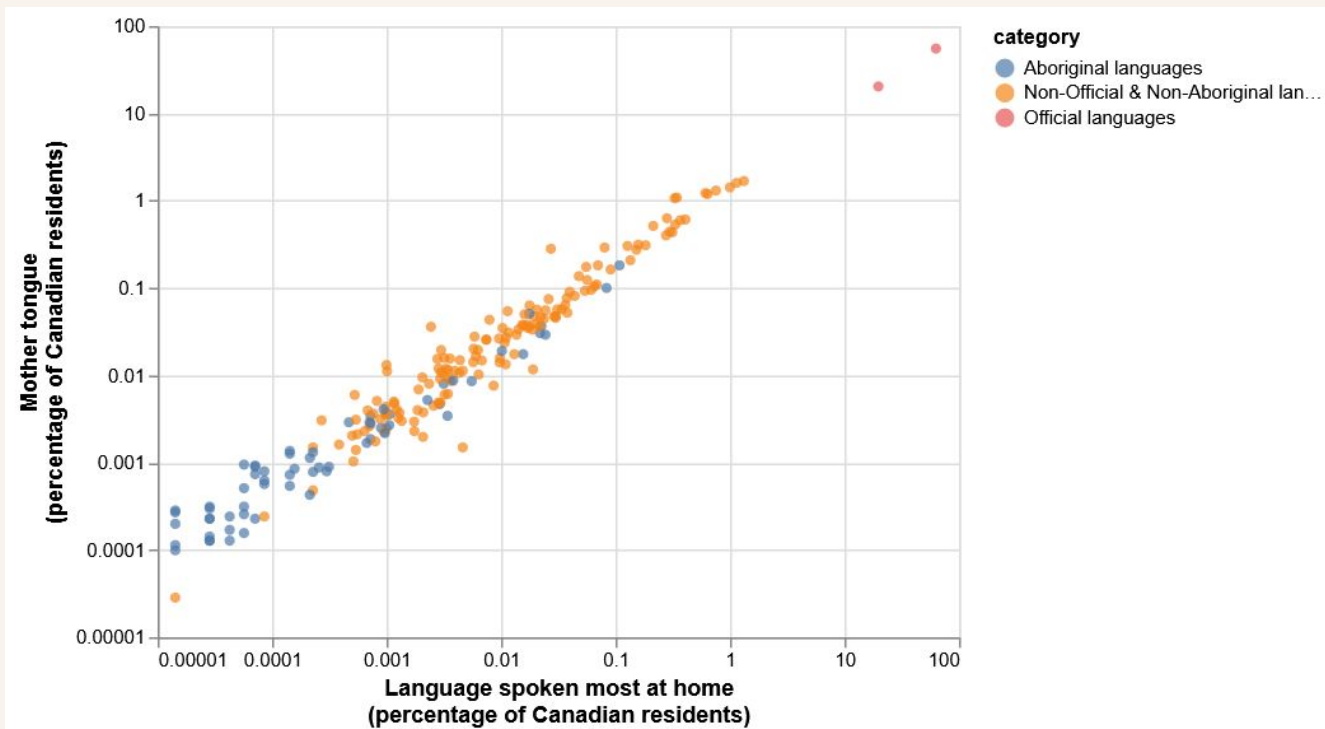


Logarithmic Scaling

```
can_lang_plot_log_revised = alt.Chart(can_lang).mark_circle().encode(  
    x=alt.X("most_at_home")  
        .scale(type="log")  
        .title(["Language spoken most at home", "(number of Canadian residents)"]),  
    y=alt.Y("mother_tongue")  
        .scale(type="log")  
        .title(["Mother tongue", "(number of Canadian residents)"]),  
    .configure_axis(titleFontSize=12)
```

Scatter Plots with Categories

❏ We can use color to show categories within the data

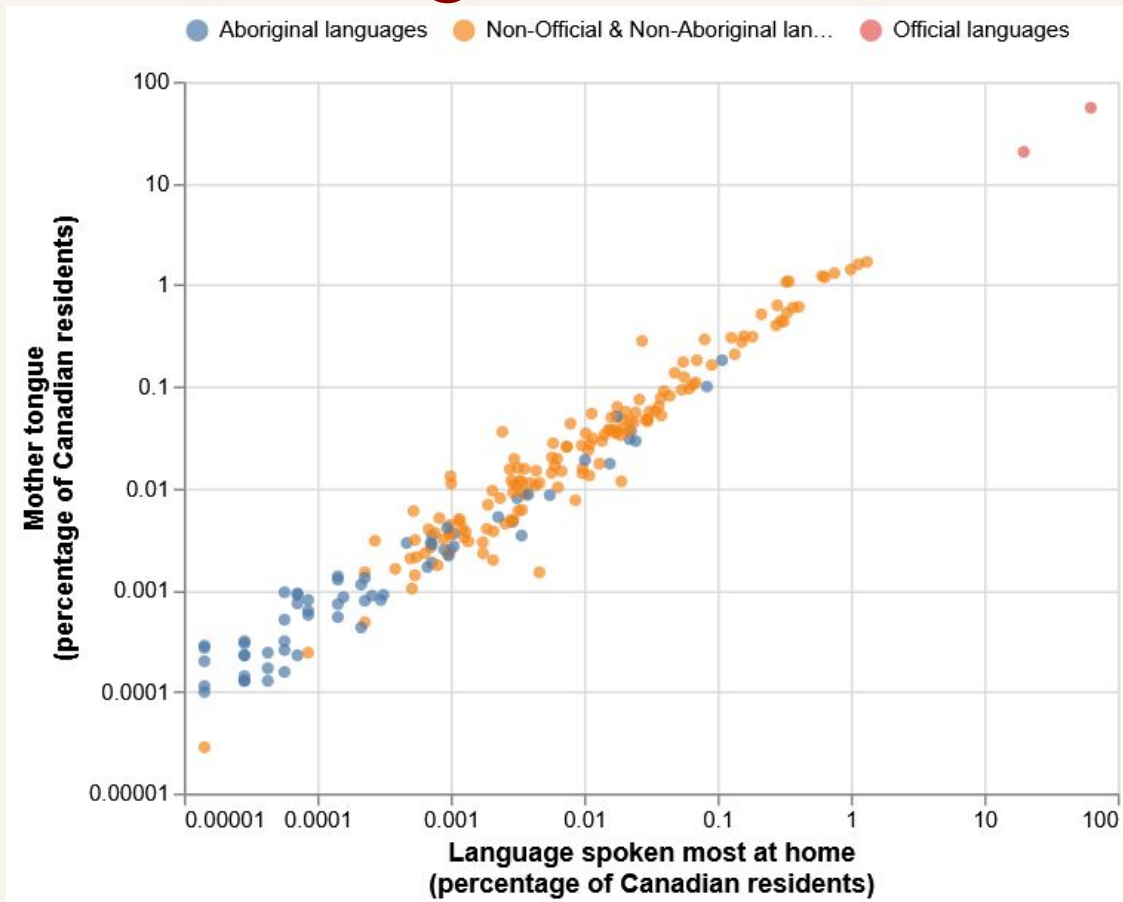


Scatter Plots with Categories

```
can_lang_plot_category=alt.Chart(can_lang).mark_circle().encode(  
    x=alt.X("most_at_home_percent")  
        .scale(type="log")  
        .axis(tickCount=7)  
        .title(["Language spoken most at home", "(percentage of Canadian residents)"]),  
    y=alt.Y("mother_tongue_percent")  
        .scale(type="log")  
        .axis(tickCount=7)  
        .title(["Mother tongue", "(percentage of Canadian residents)"]),  
    color="category"  
) .configure_axis(titleFontSize=12)
```

Scatter Plots with Categories

❏ We can also
move the
legend

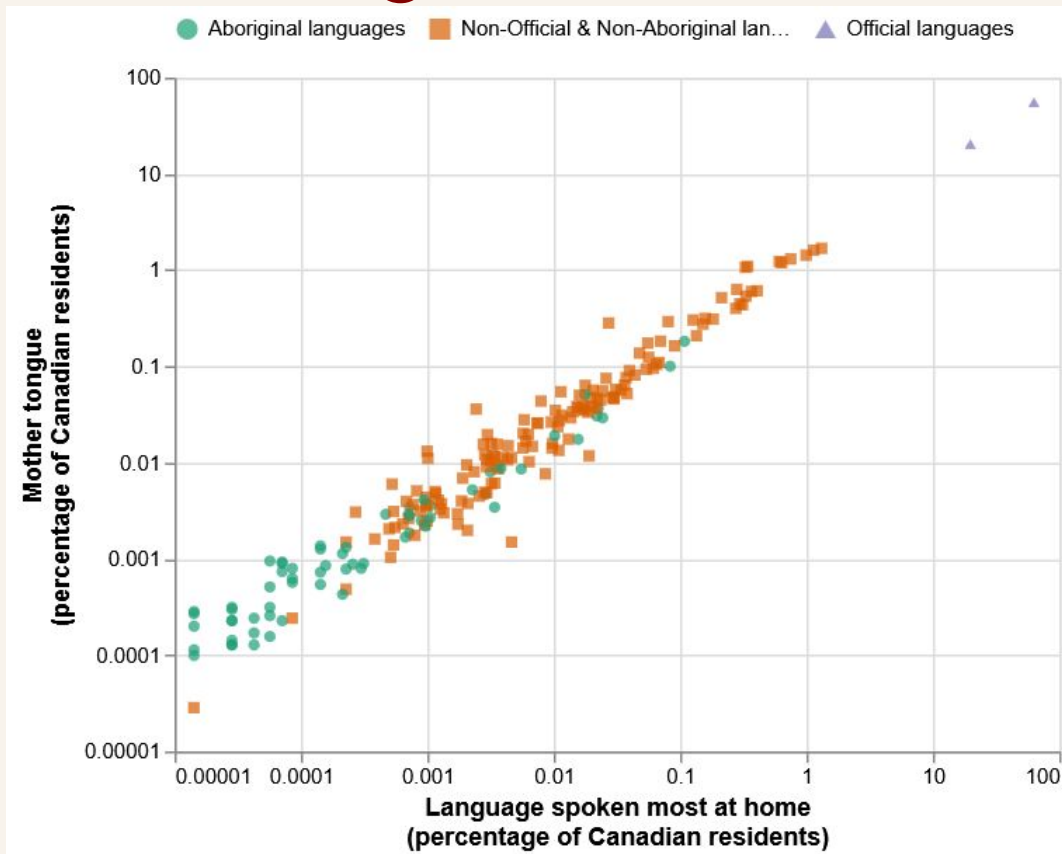


Scatter Plots with Categories

```
can_lang_plot_legend = alt.Chart(can_lang).mark_circle().encode(  
    x=alt.X("most_at_home_percent")  
        .scale(type="log")  
        .axis(tickCount=7)  
        .title(["Language spoken most at home", "(percentage of Canadian residents)"]),  
    y=alt.Y("mother_tongue_percent")  
        .scale(type="log")  
        .axis(tickCount=7)  
        .title(["Mother tongue", "(percentage of Canadian residents)"]),  
    color=alt.Color("category")  
        .legend(orient="top")  
        .title("")  
) .configure_axis(titleFontSize=12)
```

Scatter Plots with Categories

□ We can also change the color scheme and shape representing each category

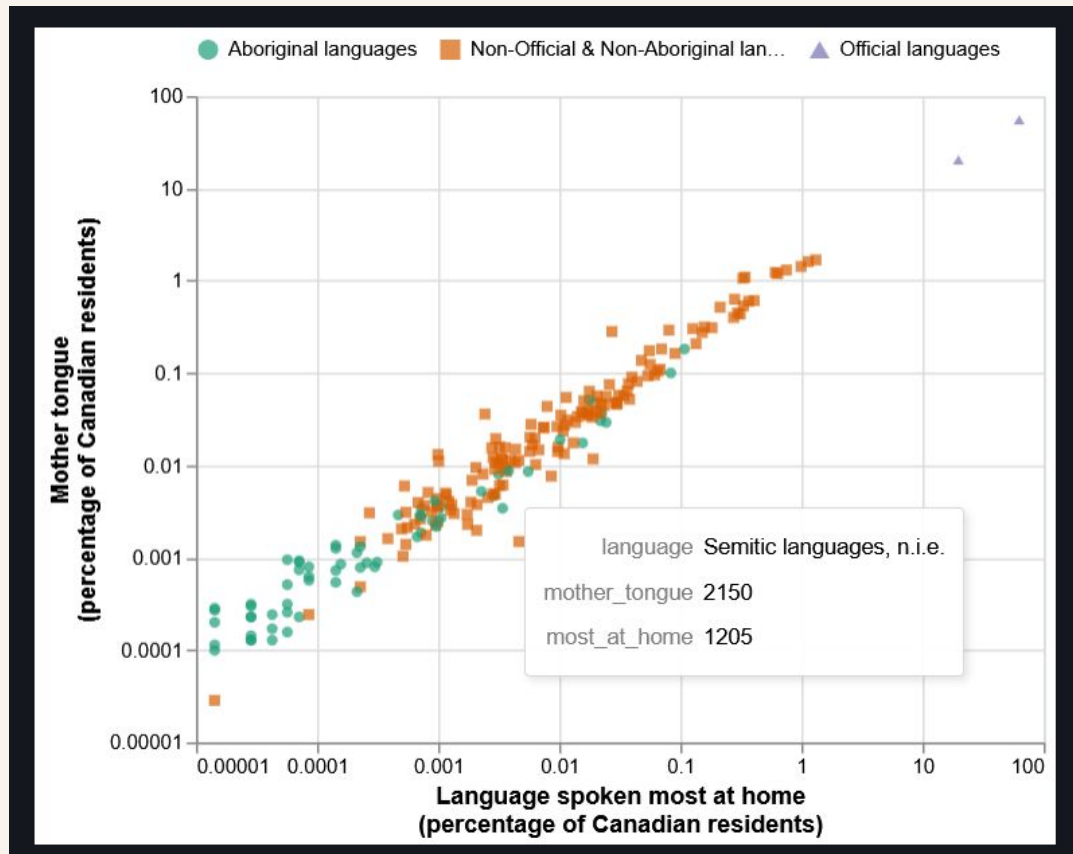


Scatter Plots with Categories

```
can_lang_plot_theme = alt.Chart(can_lang).mark_point(filled=True).encode(  
    x=alt.X("most_at_home_percent")  
        .scale(type="log")  
        .axis(tickCount=7)  
        .title(["Language spoken most at home", "(percentage of Canadian residents)"]),  
    y=alt.Y("mother_tongue_percent")  
        .scale(type="log")  
        .axis(tickCount=7)  
        .title(["Mother tongue", "(percentage of Canadian residents)"]),  
    color=alt.Color("category")  
        .legend(orient="top")  
        .title("")  
        .scale(scheme="dark2"),  
    shape="category"  
) .configure_axis(titleFontSize=12)
```


Tooltip

❏ We can add a Tooltip to make a visualization interactive



Tooltip

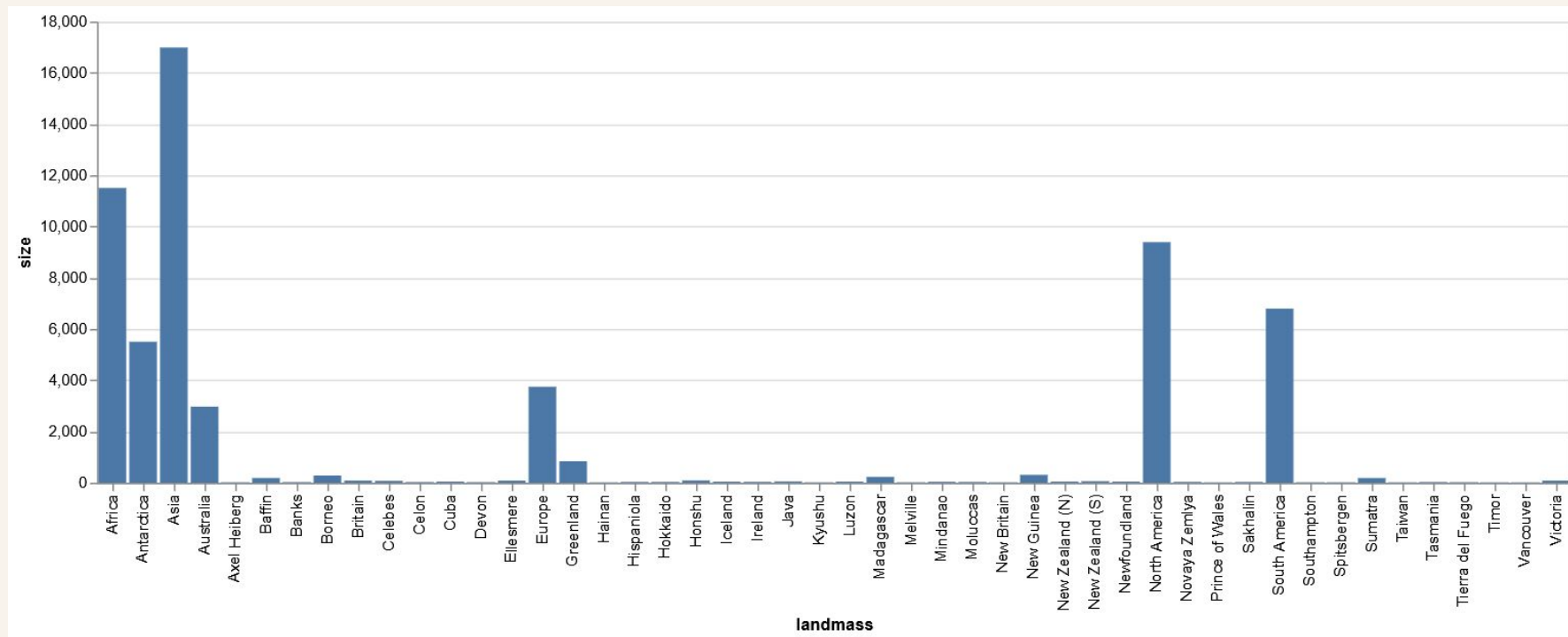
```
can_lang_plot_tooltip = alt.Chart(can_lang).mark_point(filled=True).encode(  
    x=alt.X("most_at_home_percent")  
        .scale(type="log")  
        .axis(tickCount=7)  
        .title(["Language spoken most at home", "(percentage of Canadian residents)"]),  
    y=alt.Y("mother_tongue_percent")  
        .scale(type="log")  
        .axis(tickCount=7)  
        .title(["Mother tongue", "(percentage of Canadian residents)"]),  
    color=alt.Color("category")  
        .legend(orient="top")  
        .title("")  
        .scale(scheme="dark2"),  
    shape="category",  
    tooltip=alt.Tooltip(["language", "mother_tongue", "most_at_home"])  
) .configure_axis(titleFontSize=12)
```

Bar Plots

- ❑ The height of each bar represents the value of an amount (a size, count, proportion, percentage, etc)
- ❑ Bar plots should generally not be used to display mean or median values, as they hide important information about the variation of the data
 - ❑ Instead, use a histogram

Bar Plots

❏ We use `mark_bar` to create a bar chart



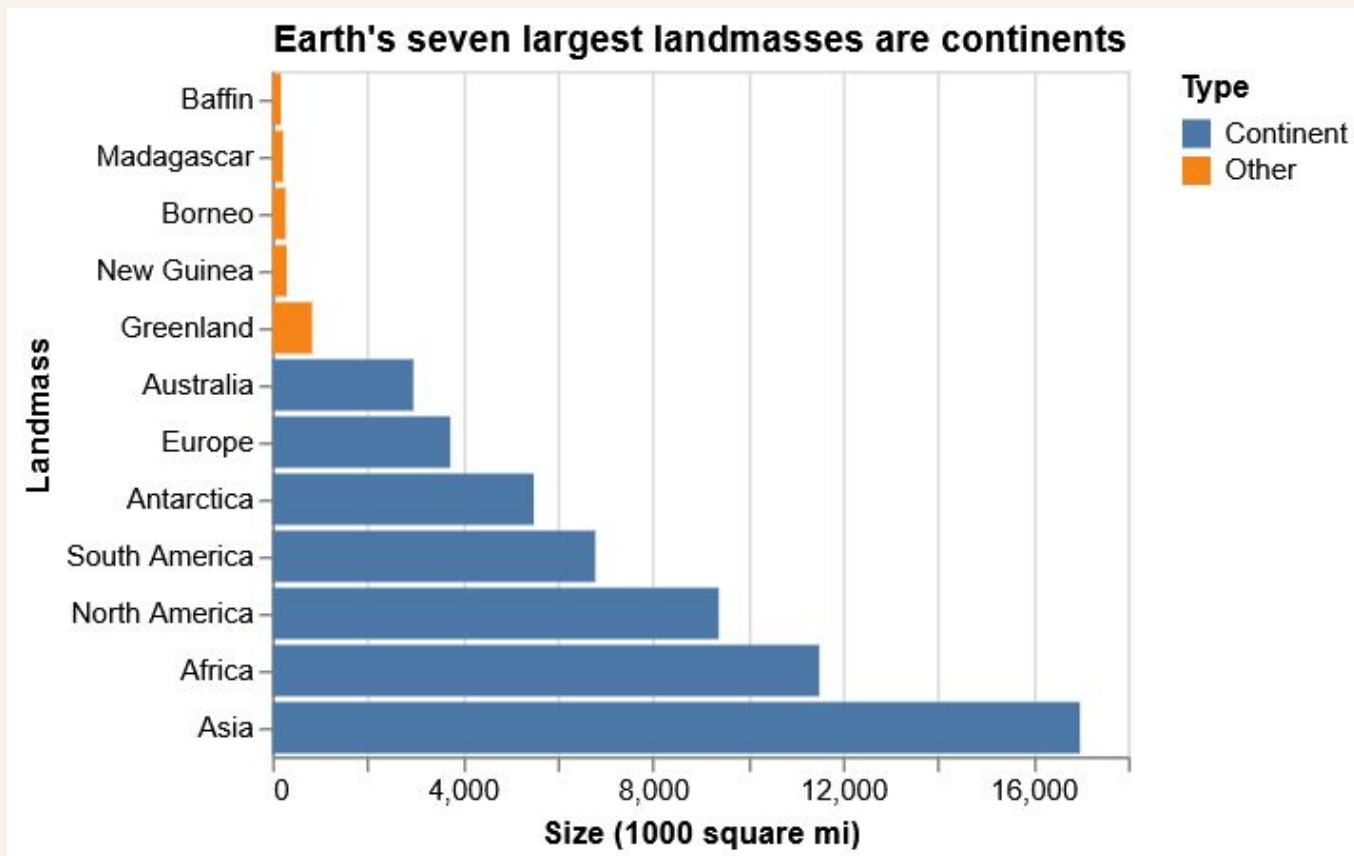
Bar Plots

```
islands_bar = alt.Chart(islands_df).mark_bar().encode(  
    x=alt.X("landmass"),  
    y=alt.Y("size")  
)
```

Bar Plots

- ❑ We can improve our bar plot by organizing the bars from largest to smallest
- ❑ We can also use the **nlargest** function to look at only the N largest values
- ❑ Like scatter plots, we can use color to categorize the bars in a bar plot

Bar Plots

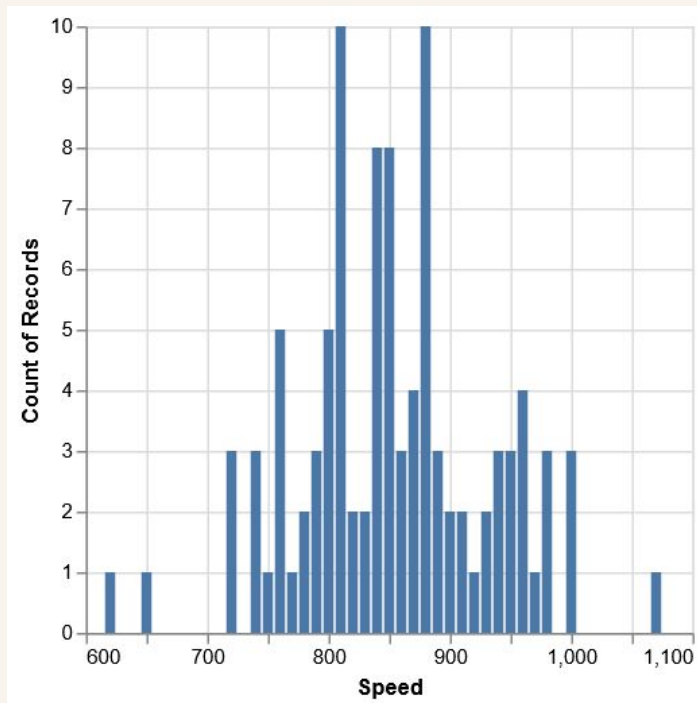


Bar Plots

```
islands_top12 = islands_df.nlargest(12, "size")
islands_plot_sorted = alt.Chart(
    islands_top12,
    title="Earth's seven largest landmasses are continents"
).mark_bar().encode(
    x=alt.X("size").title("Size (1000 square mi)"),
    y=alt.Y("landmass").sort("x").title("Landmass"),
    color=alt.Color("landmass_type").title("Type")
)
```

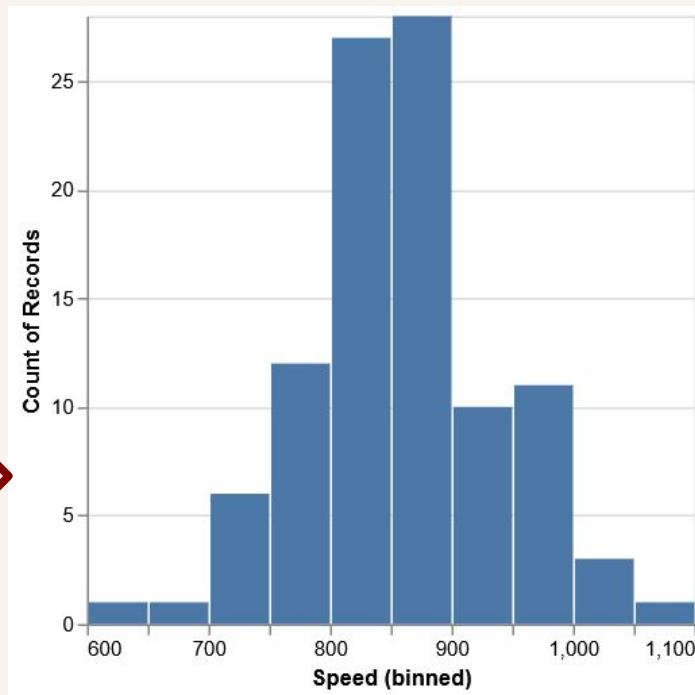

Histograms

- ❏ A histogram is a bar plot where the values are **binned** so we can more easily see the data's distribution



Bar
Plot

Histo-
gram



Histograms

```
morley_hist = alt.Chart(morley_df).mark_bar().encode(  
    x=alt.X("Speed").bin(),  
    y=alt.Y("count()")  
)
```

❏ We can change the number of bins with **maxbins**

```
morley_hist = alt.Chart(morley_df).mark_bar().encode(  
    x=alt.X("Speed").bin(maxbins=30),  
    y=alt.Y("count()")  
)
```

Chart Layers

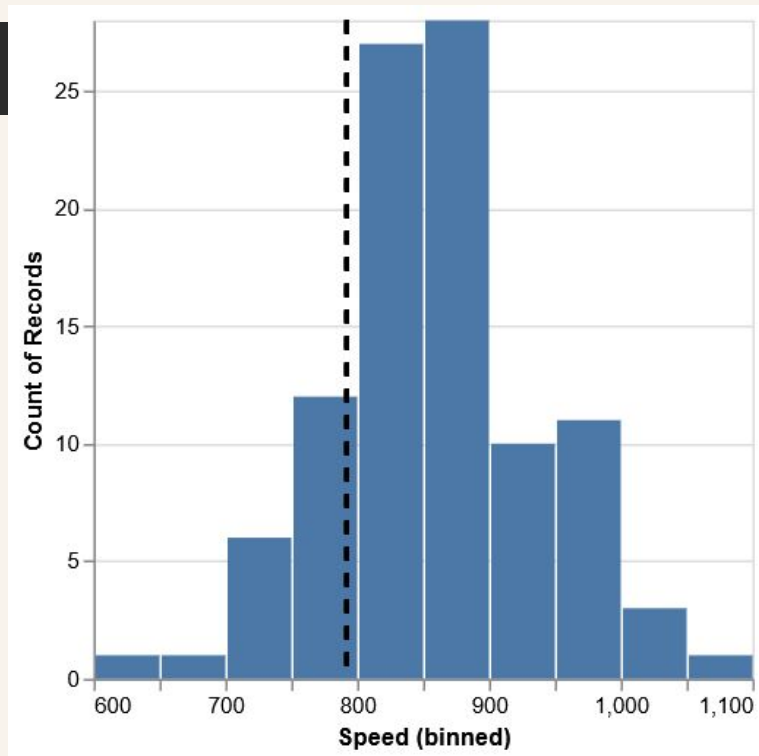
- ❑ Altair charts can be layered on top of one another to make more complex visualizations
- ❑ We can add a vertical or horizontal line to a visualization at some specific value
- ❑ Here, we make a new plot (with the same dataframe) and use **mark_rule** to create a vertical line

```
v_line = alt.Chart(morley_df).mark_rule(strokeDash=[6],  
size=1.5).encode(  
    x=alt.datum(792.458)  
)
```

Chart Layers

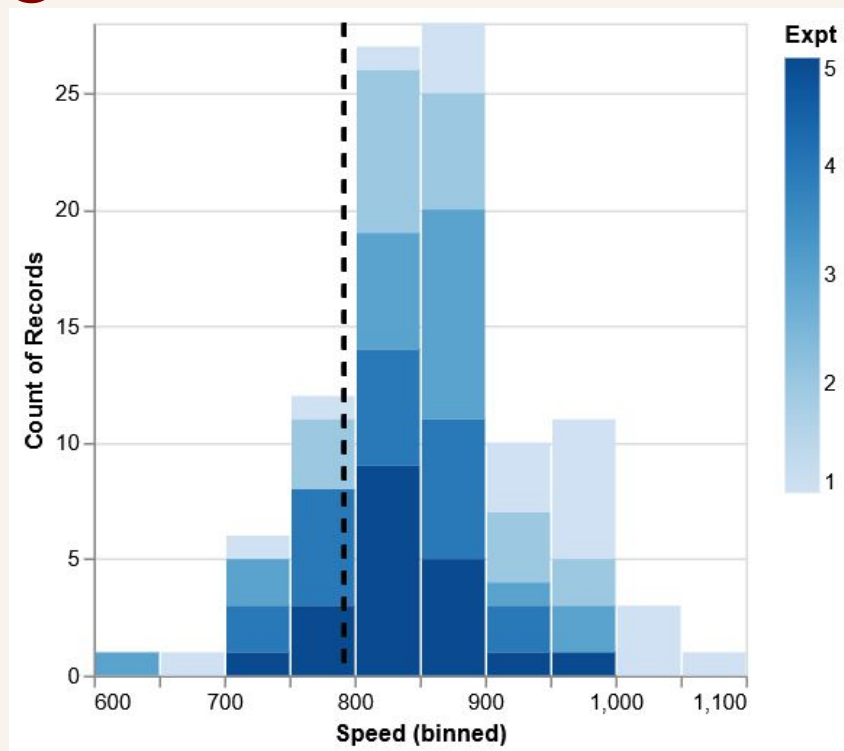
❏ Then, we combine the two layers with +

```
morley_hist_line = morley_hist + v_line
```



Troubleshooting Categories

- ❏ If we want to use a variable with integer data as a category, we have to use slightly different code



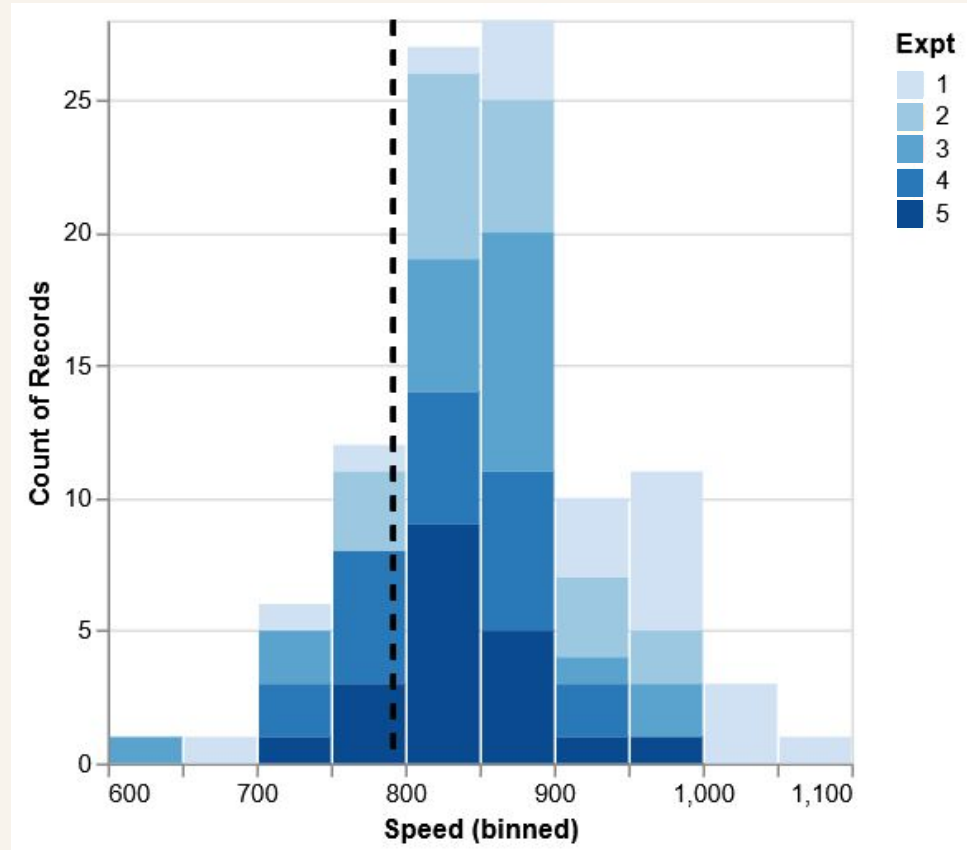
Troubleshooting Categories

- ❑ We need to specify that the category variable is **nominal** (i.e., categorical) with the suffix **:N**

```
morley_hist_categorical =  
alt.Chart(morley_df).mark_bar().encode(  
    x=alt.X("Speed").bin(),  
    y=alt.Y("count()"),  
    color="Expt:N"  
)
```

Stacked Bars

- ❑ Better, but there's still something odd about this plot
- ❑ It's hard to know the count of records for an experiment because the bars are **stacked**



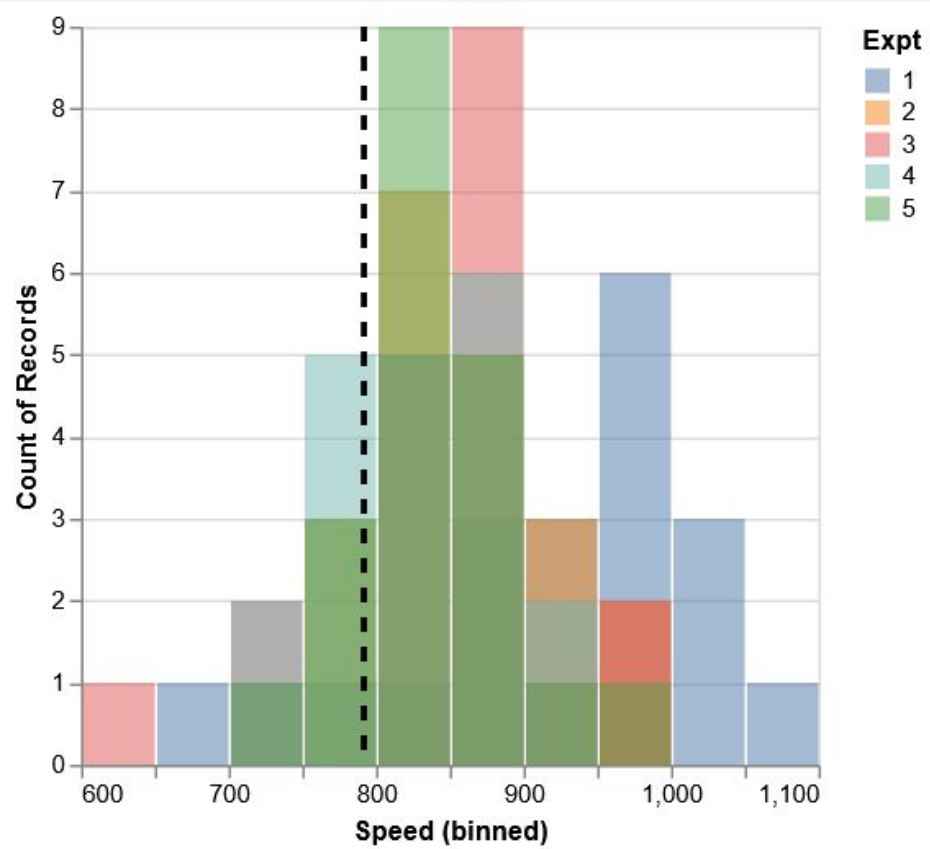
Stacked Bars

- ❑ Instead of stacking them, we can have the bars sit on top of each other
- ❑ Then we need to adjust the **opacity** to they are still visible

```
morley_hist_categorical =  
alt.Chart(morley_df).mark_bar(opacity=0.5).encode(  
  x=alt.X("Speed").bin(),  
  y=alt.Y("count()").stack(False),  
  color="Expt:N"  
)
```


Facets

- ❑ This plot is still a bit of a mess
- ❑ We will further improve it by dividing it into multiple plots, called **facets**

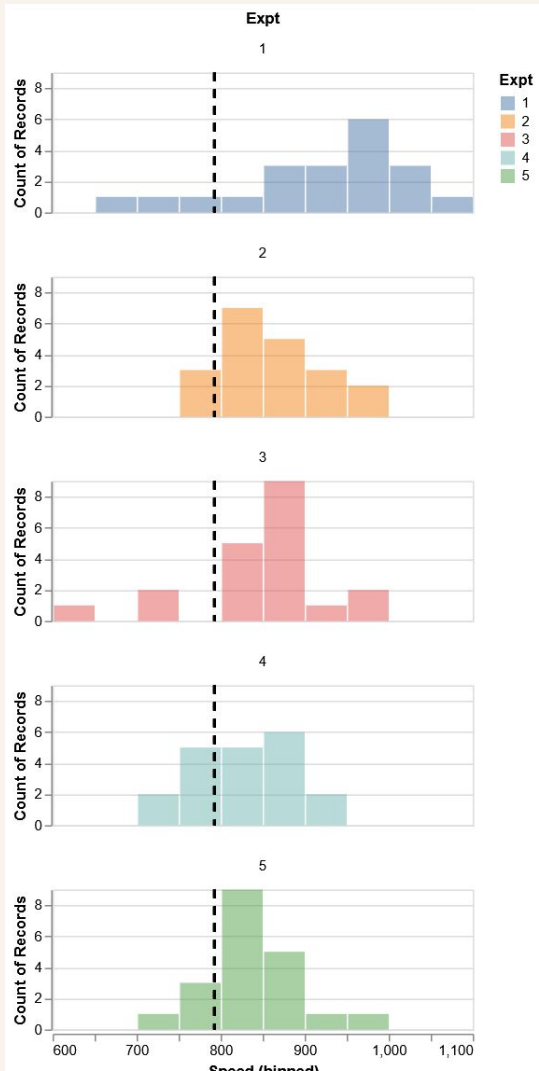


Facets

- ❑ The **facet** function specifies the variable(s) used to split the plot into subplots and how many columns there should be in a grid
- ❑ You can also specify an overall height

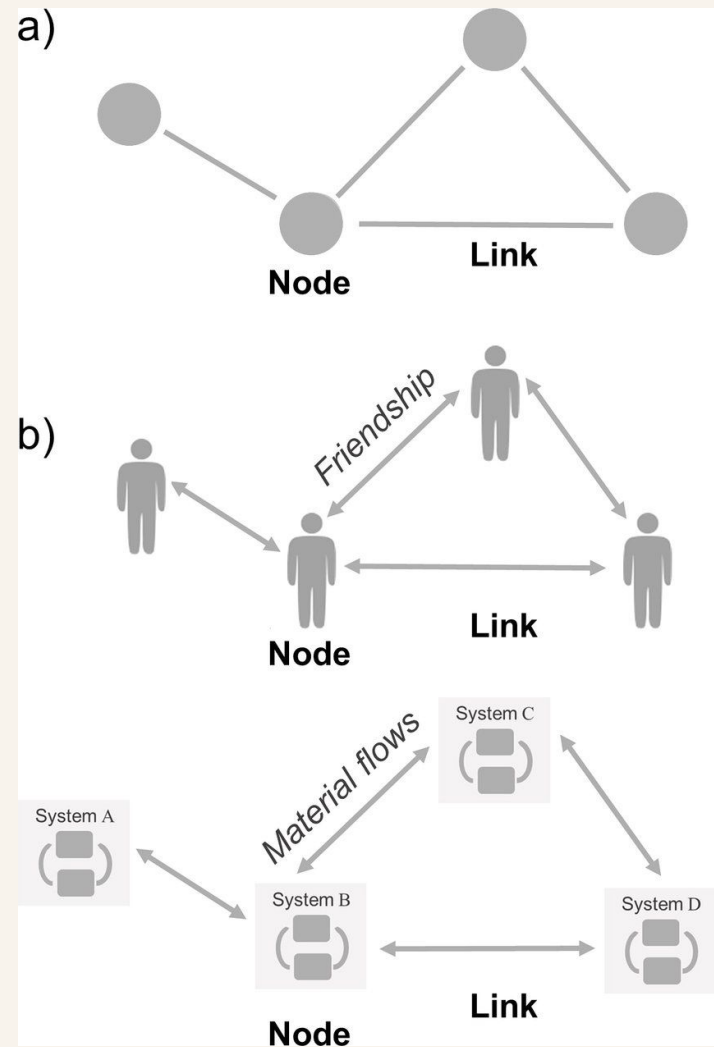
```
morley_hist_facet = morley_hist_categorical.properties(  
    height=100  
) .facet(  
    "Expt:N",  
    columns=1  
)
```

Facets



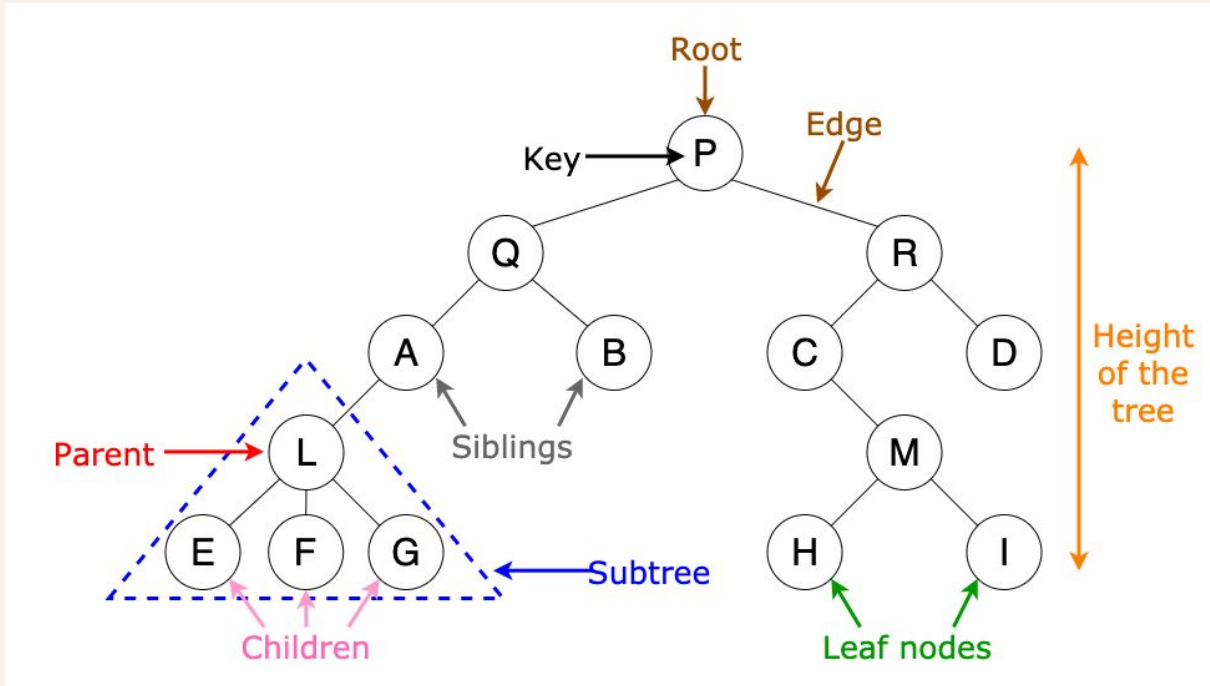
Graphs and Trees

- ❏ Graphs, also called networks, are data represented by nodes connected by links



Graphs and Trees

- ❑ A tree is just a special case of a graph where the nodes are organized hierarchically



Graphs and Trees

json

```
{
  "nodes": [
    { "id": "n1", "label": "Alice", "group": 1, "size": 20 },
    { "id": "n2", "label": "Bob", "group": 1, "size": 15 },
    { "id": "n3", "label": "Charlie", "group": 2, "size": 25 },
    { "id": "n4", "label": "David", "group": 2, "size": 10 }
  ],
  "links": [
    { "source": "n1", "target": "n2", "type": "friend", "weight": 5 },
    { "source": "n2", "target": "n3", "type": "coworker", "weight": 2 },
    { "source": "n3", "target": "n4", "type": "family", "weight": 9 },
    { "source": "n4", "target": "n1", "type": "friend", "weight": 1 }
  ]
}
```

Graphs and Trees

[Learn Python Altair: Networks](#)

Explaining Visualizations

- ❑ **Tell a story**
- ❑ Establish the setting and scope, and describe why you did what you did
- ❑ Pose the question your visualization answers and justify why its important
- ❑ Answer the question using your visualization, making sure you describe *all* aspects of the visualization (including the axes)

Explaining Visualizations

- ❑ **Trends (lines):** Is it linear or nonlinear? Increasing, decreasing, or neither? Is there a periodic oscillation? Is the trend noisy or smooth?
- ❑ **Distributions (scatters, histograms):** How spread out are they data? Where are they centered? Are there any obvious clusters or subgroups (multiple bumps in the histogram)?

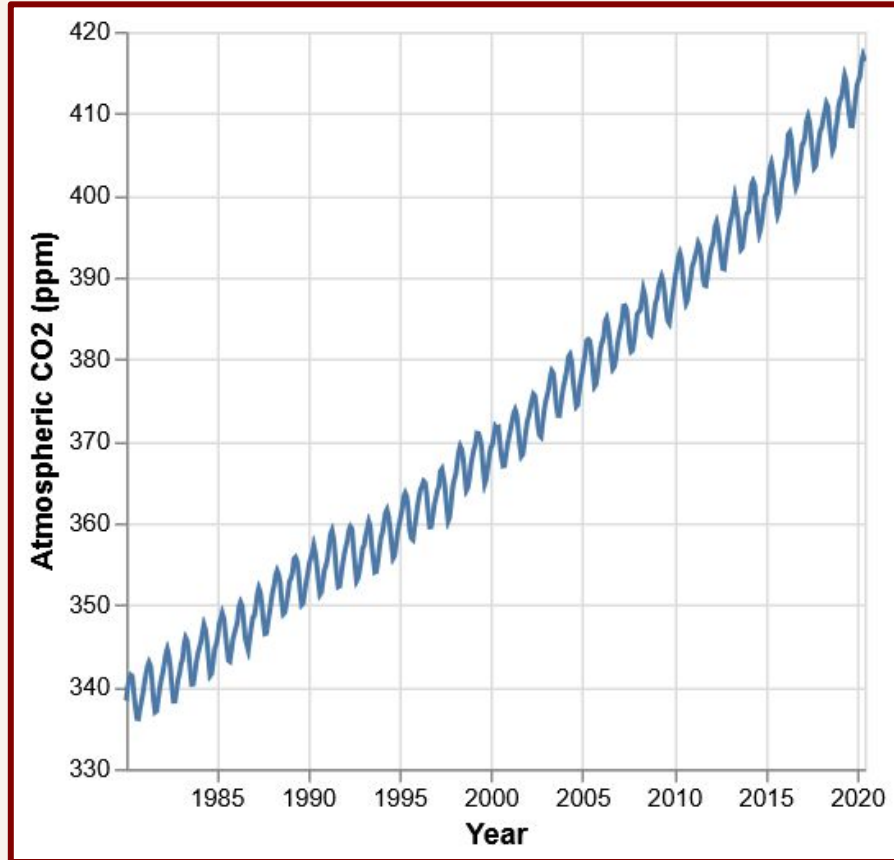
Explaining Visualizations

- ❑ **Distributions of two variables (scatters):** is there a strong, weak, or no relationship between the variables?
- ❑ **Amount (bars):** How large are the bars relative to one another? Are there patterns in different groups of bars?
- ❑ Finally, summarize your findings, and use them to motivate the next part of your presentation (if applicable)

Explaining Visualizations: Example 1

Mauna Loa Atmospheric CO Measurements: (1) Many current forms of energy generation and conversion—from automotive engines to natural gas power plants—rely on burning fossil fuels and produce greenhouse gases, typically primarily carbon dioxide (CO), as a byproduct. Too much of these gases in the Earth's atmosphere will cause it to trap more heat from the sun, leading to global warming. (2) In order to assess how quickly the atmospheric concentration of CO is increasing over time, we (3) used a data set from the Mauna Loa observatory in Hawaii, consisting of CO measurements from 1980 to 2020. We plotted the measured concentration of CO (on the vertical axis) over time (on the horizontal axis). From this plot, you can see a clear, increasing, and generally linear trend over time. There is also a periodic oscillation that occurs once per year and aligns with Hawaii's seasons, with an amplitude that is small relative to the growth in the overall trend. This shows that atmospheric CO is clearly increasing over time, and (4) it is perhaps worth investigating more into the causes.

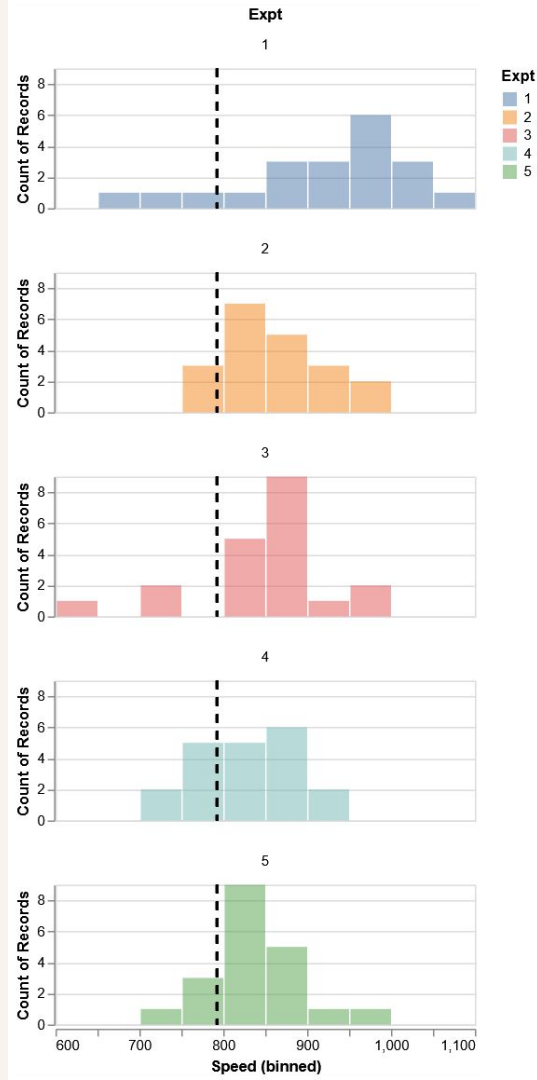
Explaining Visualizations: Example 1



Explaining Visualizations: Example 2

Michelson Light Speed Experiments: (1) Our modern understanding of the physics of light has advanced significantly from the late 1800s when Michelson and Morley's experiments first demonstrated that it had a finite speed. We now know, based on modern experiments, that it moves at roughly 299,792.458 kilometers per second. (2) But how accurately were we first able to measure this fundamental physical constant, and did certain experiments produce more accurate results than others? (3) To better understand this, we plotted data from 5 experiments by Michelson in 1879, each with 20 trials, as histograms stacked on top of one another. The horizontal axis shows the error of the measurements relative to the true speed of light as we know it today, expressed as a percentage. From this visualization, you can see that most results had relative errors of at most 0.05%. You can also see that experiments 1 and 3 had measurements that were the farthest from the true value, and experiment 5 tended to provide the most consistently accurate result. (4) It would be worth further investigating the differences between these experiments to see why they produced different results.

Explaining Visualizations: Example 2



Saving Visualizations

- ❑ **Raster images** are 2-D grids of pixels: JPEG, PNG, BMP, TIFF
- ❑ **Vector images** are collections of mathematical objects (lines, surfaces, shapes, curves) that are redrawn using the mathematical formulas when the image is displayed: SVG, EPS
- ❑ A raster image always takes the same amount of time/space to load, but a vector image takes more time/space if it is more complex
- ❑ Vector images allow for infinite zoom without distortion

Saving Visualizations

```
faithful_scatter_labels.save("img/viz/faithful_plot.png")  
faithful_scatter_labels.save("img/viz/faithful_plot.svg")
```


Homework Reminder

- ❑ Finish up Phase 2 of the Data Collection Project
- ❑ Meet with your partner to work on Phase 3
- ❑ Presentations will be next week



Big timers



MINS



MINS



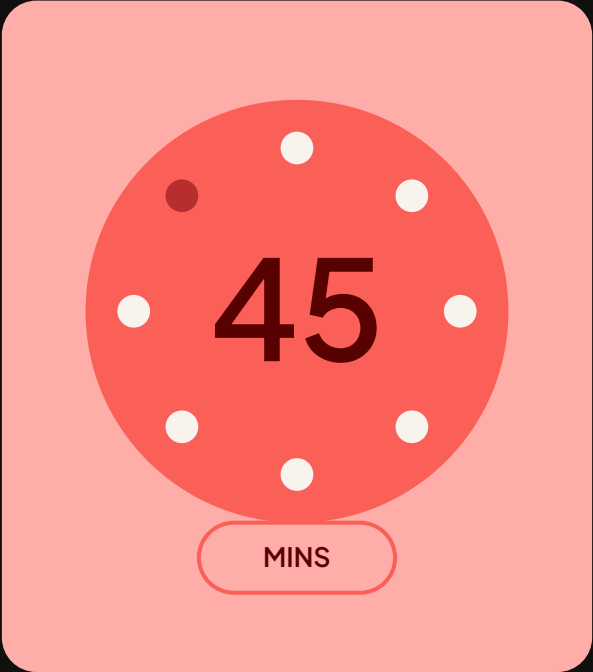
MINS

Big timers



30

MINS



45

MINS



01

HS